

最短路算法及扩展

主讲：黄凤林

问题的提出

- ✧ 给定一个图 $G(V, E)$, V 是顶点集合, E 是边集合, 每一条边有一个权值 c , 给定一个起始点 S 和终止点 D 。
- ✧ 求从 S 出发走到 D 的权值最小路径, 即为最短路径。



问题的类型

- ▶ 单源最短路
- ▶ 所有点对最短路
- ▶ 单位权值
- ▶ 非负权值
- ▶ 负权值





➔ 单源最短路径

Single Source Shortest Path

▶ 问题描述:

求给定起点到其他所有点的最短路径

▶ 相关算法:

Dijkstra

Bellman-Ford

SPFA(队列优化的Bellman-Ford)



→ 迷宫问题

典型的搜索问题，直接**BFS**即可

其实也可看做单源最短路问题

对应关系：

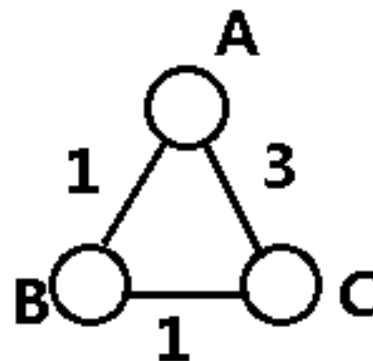
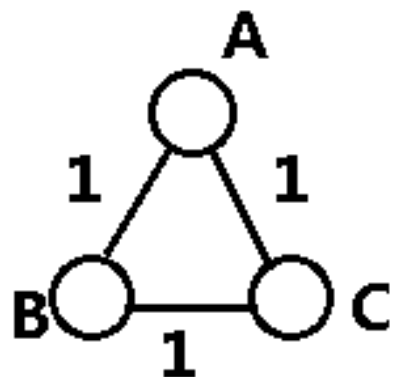
起点终点对应

迷宫的每个格子对应图中的点

每种移动对应图中的边，且边权值为1

思考：边权不为1的情况能**BFS**解决吗？为什么？

→ 一个例子



求图中从A到C的最短路

思考：为什么BFS不行？

因为 $\text{dis}[c] > \text{dis}[b] + \text{len}(b, c)$ 怎么解决？

$\text{dis}[c] = \text{dis}[b] + \text{len}(b, c)$ 松弛操作



→ Dijkstra算法

设置一顶点集合 S ，从源点 s 到集合中的顶点的最终路径的权值均已确定

对于不在 S 中的点来说，有一个距离的估计值，初始时为正无穷，会随松弛操作而改变

反复选取不在 S 中且估计值最小的点 u ，将其加入 S ，并用它去松弛其他点

→ Dijkstra算法

时间复杂度? $O(V^2)$

算法复杂度的瓶颈? 如何快速找最小值

堆 OR 优先队列!

时间复杂度? $O(E \log V)$

Sea Base Exploration (CDOJ 1485)

状态压缩+优先队列的BFS

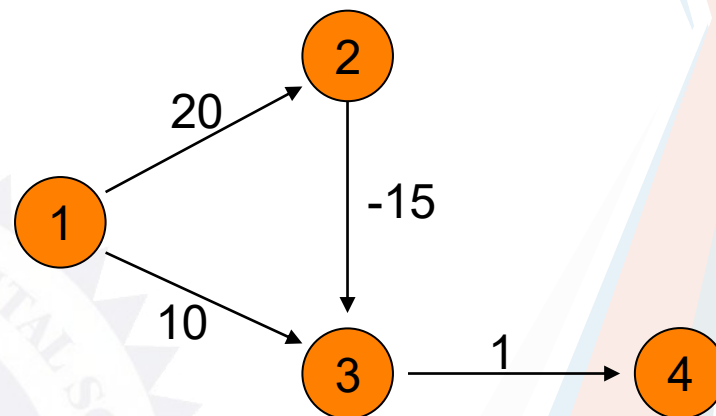
其实就是用dijkstra解决的最短路!

→ Dijkstra算法的局限性

dijkstra能处理负边权么？

有图有真相 不能！

事实上



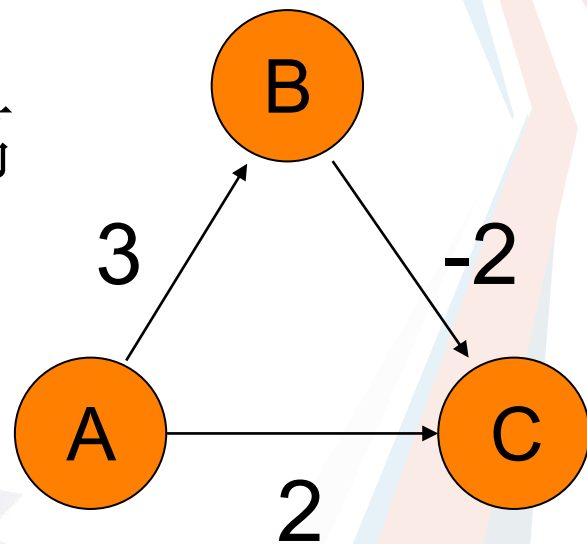
如果负边只存在于源点s连出去的边，
dijkstra还是可以做的

那么，对于一般的有负边权的呢？

➔ Dijkstra的局限性

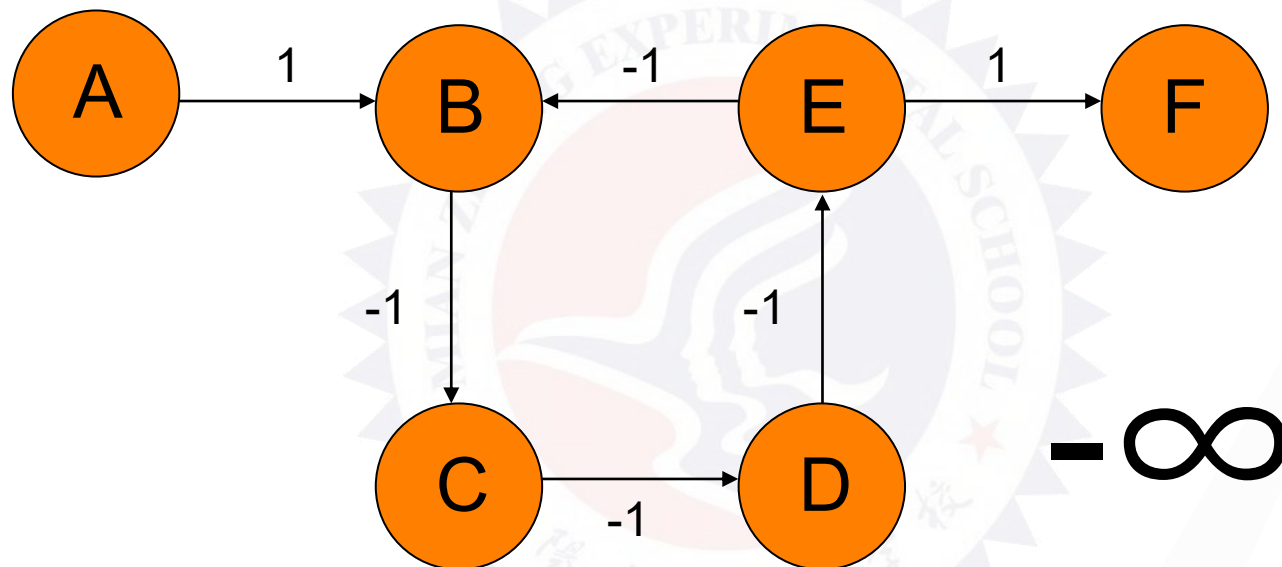
求解右图A 至其他点的最短距离
算法步骤：

- 1) 标记点A
 - 2) $\text{Dist}[C]=2$ 最小，标记点C
 - 3) $\text{Dist}[B]=3$ 最小，标记点B
- 结束
但是 $\text{ShortestDist}[C]=1$



➔ Dijkstra的局限性

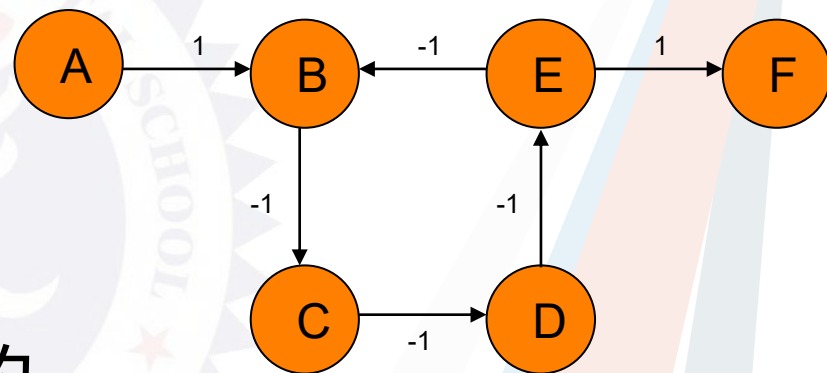
下图中，A至F的最短路径长度是多少？



$-\infty$

➔ Dijkstra的局限性

- 如果利用Dijkstra算法求解
- 标记点A, $\text{Dist}[B]=1$, 标记点B
- $\text{Dist}[C]=0$, 标记点C
- $\text{Dist}[D]=-1$, 标记点D
- $\text{Dist}[E]=-2$, 标记点E
- $\text{Dist}[F]=-1$, 标记点F
- 所求得的距离并不是最短的



➔ 错误结果的原因

Dijkstra的缺陷就在于它不能处理负权边：

Dijkstra对于标记过的点就不再进行更新了，所以即使有负权导致最短距离的改变也不会重新计算已经计算过的结果。

我们需要新的算法——**Bellman-Ford**

定理：在一个边权可正可负的图中，若存在从源点到某个结点的一条最短路径，那么这条最短路径一定不含环。

证明：在边权可正可负的图中，假设某条最短路径包含环路。考察环上所有边权之和有3种情况：正权环、零权环、负权环。

对于正权环、零权环，去掉环路后，整条路径的长度会更短。

这与它是最短路径矛盾。故此时最短路径不包含环。

对负权环，则沿环路走一圈最短距离会减少一次。故此时不存在最短路径。

→ 最短路的重要性质

一个图在经过最短路 (Dijkstra或其它)算法后
在最短路径上的边(u,v)满足:

$$\text{dist}[v] = \text{dist}[u] + w(u,v)$$

其它(非最短路径上的)边(x,y)满足:

$$\text{dist}[y] \geq \text{dist}[x] + w(x, y)$$

我们把图所具备的这种状态称为“收敛”。显然，
不含负权边的图一定会收敛。

→ Bellman-Ford算法思想

- Bellman-Ford算法基于动态规划，反复用已有的边更新最短距离——**松弛**
- 当尚未收敛时，图中存在这样的边 (u,v) 满足： $\text{dist}[v] > \text{dist}[u] + w(u,v)$ 时，就可以更新为： $\text{dist}[v] = \text{dist}[u] + w(u, v)$
- 反复利用上式对 dist 数组进行松弛，如果没有负权环的话，应当会在有限次松弛之后，所以结点的最短距离就求出。**那么上限是多少遍呢？**
- **换一个角度看问题：任何一条最短路径，要么经过0个中间点，要么经过1个中间点，要么经过2个中间点.....**
- **有 n 个结点的图中，一条最短路径最多有 $n-2$ 个中间点(不算起点和终点)**

➔ Bellman-Ford算法思想

- ◆ 也就是说，源点到其它各结点的最短路径，即使在有负边时，也具有如下性质：最多经过 $n-2$ 个中间点。
- ◆ 初始时，源点的最短距离可以确定，即 $\text{dist}[s]=0$ 。 $\text{dist}[v]=\text{INF}(v \neq s)$
- ◆ 当对每条边进行第1遍松弛后，将求出至多经过0个中间点的最短路径，对每条边进行第2遍松弛后，将求出至多经过1个中间点的最短路径，……
- ◆ 如果没有负权环，那么经过第 $n-1$ 遍松弛操作后，就可以求出至多经过 $n-2$ 个中间点的最短路。
- ◆ 如果有负权环，那么第 n 遍松弛操作仍然会成功。这时，图仍不会收敛。图中仍存在边满足 $\text{dist}[v] > \text{dist}[u] + w(u,v)$ 。这时，最短路径不存在。

→ Bellman-Ford算法

- 先来看伪代码:

```
bool Bellman-Ford(int s){  
    for(i = 1; i <= v; ++i) d[i] = INF;所有点的d[]值置为无穷大  
    d[s] = 0;源点d[]值为0  
    for(i = 1; i < v; ++i)  
        for each edge(u, v, e)  
            relax(u, v, w);进行v - 1次松弛操作  
    for each edge(u, v, e)  
        if(d[v] > d[u] + e) return false;  
    如果松弛操作结束后仍有可继续松弛的边, 则图中存在负环  
    return true;  
}
```

→ Bellman-Ford算法

正确性的证明

- 如果原图中不存在负环：

从 s 可达的任意点 v 的最短路至多经过 $v - 1$ 条边。

对于这 $v - 1$ 条边，第一次松弛后第一条边不可再松弛。第二次松弛后第二条边不可再松弛，...这样便证明了算法在没有负环的图上的正确性。

- 如果原图中存在负环：

显然对于负环中的任何一个点，它的最短路会是负无穷，所以松弛操作不会停止。算法返回**false**。

→ Bellman-Ford算法

时间复杂度

两个for循环 $O(VE)$

相当慢！

做了很多无用功！

一个优化：

每次只用距离减小的点去更新其他点

如何实现？ *队列！*

→ 优化—SPFA算法

- Bellman-Ford算法之所以效率不高，是因为在每遍循环中要去枚举整个边集，而实际上有些边并不会松弛成功。
- 如果把Bellman-Ford算法看成一个类似BFS这样的层次遍历的过程。当某个点 v 被松弛成功了， v 才可能去松弛后续结点。因此把 v 加入一个FIFO队列。这样就会不断减少边的枚举量。
- 一个点出队后可能再次入队。因为它可能再次被松弛。这就是Bellman-Ford算法的队列优化，称为SPFA(Shortest Path Faster Algorithm)
- 初始时将源加入队列。每次从队列中取出一个元素，并对所有与它相邻的点进行松弛，当某个点松弛成功，若它不现队中，将其入队，否则不入队。直到队列为空时算法结束。



→ SPFA算法

整理一下新算法的思路：

Step 1:初始时所有点 $d[]$ 值置 INF ，源点 $d[]$ 为0。将源点放进队列。

Step 2:当队列不为空时每次从队列中取出队首，对队首的每条边进行松弛。将松弛后 $d[]$ 值改变并且不在队列中的点加入队列。

这就是SPFA算法！



→ SPFA算法

两点说明：

时间复杂度 最坏： $O(VE)$

一般： $O(kE)$

如何判负环？

对每个点记录一个num值，表示被更新了多少次，如果某个点被更新的次数超过 $n-1$ 次，则有负环

➔ SPFA算法的效率

- 时间复杂度一般认为是 $O(kE)$
- 其中 k 是一个较大的常数，不好估计，但是可以看出SPFA算法效率应当是很高的
- 经验表明Dijkstra算法的堆优化要比SPFA快，但SPFA比普通的Dijkstra算法快。而SPFA算法可以处理负权的问题，而且比Dijkstra算法的堆优化的代码要容易实现，因此SPFA是一个很好的算法。
- 注意：后面这段SPFA代码不能处理存在负权环的情况。当存在负权环里，松弛操作一直会成功，队列永不会空，就会陷入死循环。这时，可以设置一个结点入队次数的计数器。当一个结点入队达到 n 次后，就表明图中存在负权环，可以结束算法了。



SPFA代码

```
void spfa(int s){
    for(int i=1;i<=n;i++) dist[i]=inf;
    dist[s]=0; q.push(s); vis[s]=true;
    while(!q.empty()){
        int u=q.front();q.pop();vis[u]=false;
        for(int i=head[u];i>0;i=edge[i].next){
            int v=edge[i].to;
            if(dist[v]>dist[u]+edge[i].w){
                dist[v]=dist[u]+edge[i].w;
                if(!vis[v]){
                    q.push(v);vis[v]=true;
                }
            }
        }
    }
}
```


➔ 所有点对最短路

▶ 问题描述:

求所有点到其他点的最短路径

▶ 相关算法:

V次dijkstra

$O(V^3)$ or $O(EV\log V)$

V次Bellman_Ford

$O(V^2E)$

V次SPFA

$O(kVE)$

1次floyd!

$O(V^3)$

→ Floyd算法

先看代码：

```
for(k=1;k<=V;k++)  
for(i=1;i<=V;i++)  
for(j=1;j<=V;j++)  
if(dis[i][j]>dis[i][k]+dis[k][j])  
dis[i][j]=dis[i][k]+dis[k][j];
```

实际上是一个精巧的DP！



→ Floyd算法

DP过程:

设 $\text{dis}[i][j][k]$ 表示从 i 到 j 的路径中，经过的点的编号不超过 k 的最短路

边界条件: $\text{dis}[i][j][0] = \text{dis}[i][j]$

转移方程:

$$\text{dis}[i][j][k] = \text{Min}(\text{dis}[i][j][k-1], \text{dis}[i][k][k-1] + \text{dis}[k][j][k-1]) \quad (k > 0, 0 \leq i, j \leq n)$$

答案: $\text{dis}[i][j][n]$



→ Floyd算法

时间复杂度： **三重for循环** $O(V^3)$

原程序实际上是这个DP的一个精巧的实现，省略了最后一维数组

因为在第k次进行更新时，只会用 $dis[u][k]$, $dis[k][u]$ 对别的dis值进行更新，而 $dis[u][k]$ 和 $dis[k][u]$ 不会改变

→ 最短路的应用-求最小环

Floyd!

求无向图最小环

最小环是指一个图中找出一个环，使得这个环上的各边的权值之和最小。一个至少包含三个顶点的边权和最小的环。

记两点间的最短路为 $dis[i][j]$ ， $g[i][j]$ 为边 $\langle i,j \rangle$ 的权值

```
for(int k=1;k<=n;k++)
{
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            answer=min(answer,dis[i][j]+g[j][k]+g[k][i])
        }
    }
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j]);
        }
    }
}
```

answer
即为答案



→ 最短路的应用---建图

1、奇怪的电梯

题目描述

呵呵，有一天ssxyh做了一个梦，梦见了一种很奇怪的电梯。大楼的每一层楼都可以停电梯，而且第 i 层楼($1 \leq i \leq N$)上有一个数字 K_i ($0 \leq K_i \leq N$)。电梯只有四个按钮：开，关，上，下。上下的层数等于当前楼层上的那个数字。当然，如果不能满足要求，相应的按钮就会失灵。例如：3 3 1 2 5代表了 K_i ($K_1=3, K_2=3, \dots$)，从一楼开始。在一楼，按“上”可以到4楼，按“下”是不起作用的，因为没有-2楼。那么，从A楼到B楼至少要按几次按钮呢？

输入格式

输入文件共有二行，第一行为三个用空格隔开的正整数，表示 N, A, B ($1 \leq N \leq 200, 1 \leq A, B \leq N$)，第二行为 N 个用空格隔开的正整数，表示 K_i 。

输出格式

输出文件仅一行，即最少按键次数,若无法到达，则输出-1

样例输入：

5 1 5

3 3 1 2 5

样例输出：

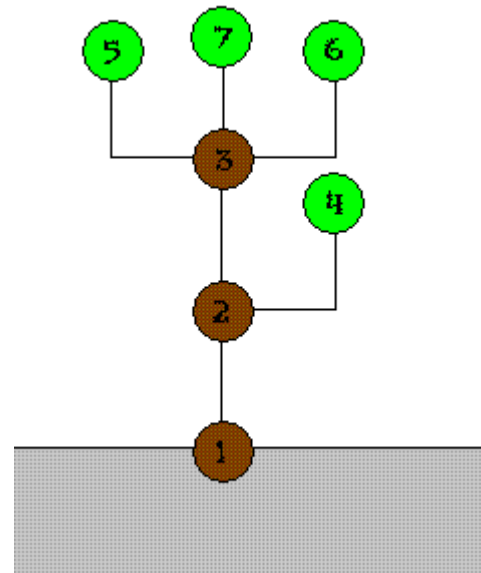
3



→ 圣诞树

【问题描述】

圣诞节到了，FireDancer 准备做一棵大圣诞树。右图为圣诞树的一个简单结构。这棵树被表示成一组被编号的结点和一些边的集合。结点从 1 到 n 编号。树的根永远是 1。每个结点都有一个自身特有的数值，称为它的重。各个结点的重可能不同。对于一棵做完的树来说，每条边都有一个价值，若设这条边 e 连接结点 i 和结点 j ，且 i 为 j 的父结点（根是最老的祖先），则该边的价值为 $(j$ 的所有子孙及它自己的重之和) $\times (e$ 的单位价值 $c_e)$ 。现在 FireDancer 想造一棵树，使得树上所有边的总价值最小，并且所有的点都在树上，因为 FireDancer 喜欢大树。





→ 圣诞树

【输入格式】

第一行两个整数 n 和 m ($0 \leq n, m \leq 50000$), 表示结点总数和可供选择的边数。

下面一行有 n 个整数, 依次表示每个结点的重。

下面 m 行, 每行有 3 个正整数 a, b, c , 表示结点 a 和结点 b 之间有一个单位价值为 c 的边可供你造树时选择。

输入中的所有数都小于 2^{16} 。

【输出格式】

若无解, 输出 “No Answer”, 否则一个整数表示造树的最小价值。

【样例输入 2】

```
7 7
200 10 20 30 40 50 60
1 2 1
2 3 3
2 4 2
3 5 4
3 7 2
3 6 3
1 5 9
```

【样例输出 2】

```
1210
```

【数据范围】

对于 50% 的数据: $n, m \leq 1000$ 。

对于 100% 的数据: $n, m \leq 5000$ 。

→ 圣诞树题解分析

- ✧ 题目要求构树的最小总价值，总价值等于每条边价值的和，而每条边的价值，又等于它所连结着的子树里的点上的总价值，乘以边的长度。
- ✧ 很显然，这条边连结的子树中每个点到根的路径中，必然包含这条边。换句话说，这棵子树里每个点都必须为这条边付出：边长*点上价值的代价。
- ✧ 那么，也不难反过来想到，为每个树上的点所付出的总代价，就是：点上价值*该点到根的路径长度。
- ✧ 显然，我们希望每个点到根的路径长度最小。
- ✧ 这就该想到最短路了，因为根结点确定为1，于是就用SPFA求单源最短路！



→ 跳格子II

题目描述

小msh长大之后，渐渐发现原来的跳格子游戏实在是太沙茶了，既没有挑战性，也没有乐趣，还经常被不明真相的群众围观.....所以，她决定修改跳格子的规则，让它看起来没那么沙茶.....

依然是在地上画一个 $n*m$ 的方格，但msh在某些格子里放上高度为1的箱子。她每次可以从一个格子跳到与它相邻的另一个格子中，如果这两个格子的高度相同，msh只需要付出1单位力气；如果这两个格子高度不同，则需要付出2单位力气。她想从某个格子跳到另一个格子，但她想用尽量少的力气。

虽然msh长大了，但她还是有点沙茶.....算了半天也没有算出来到底需要多少力气才能跳完.....那么你来帮帮她吧！

→ 最短路应用—求树的直径

✧ 问题描述:

乌托邦有 n 个城市，某些城市之间有公路连接。任意两个城市都可以通过公路直接或者间接到达，并且任意两个城市之间有且仅有一条路径(What does this imply? A tree!).

每条公路都有自己的长度，这些长度都已经测量好。小修想从一个城市出发开车到另一个城市，并且她希望经过的公路总长度最长。

请问她应该选择哪两个城市？这个最长的长度是多少？



→ 最短路的应用—建图

永恒的史诗

现在银月城中共有 n 个军团集结点，由 m 条单向道路相连接，经过每条路需要花费一定的时间。现在阿尔萨斯在银月城门口的1号集结点，只要天灾大军能攻占位于 n 号集结点的银月城大厅，银月城便会就此沦陷，阿尔萨斯决定派出几只食尸鬼军团完成这项任务。唯一的阻碍是，银月城中的一些集结点有强大的魔法屏障保护，不破坏所有保护该集结点的屏障便无法进入集结点。而每个受保护的集结点的屏障是由在其它集结点中的魔法屏障发生器所提供的，只有三月之匙能够关闭屏障。

现在，所有石像鬼军团从1号集结点出发，每个石像鬼军团拥有一把由霜之哀伤的力量复制出来的三月之匙。一旦进入某个集结点，三月之匙就会马上关闭护盾发生器，但复制的三月之匙也会失去控制屏障的力量（意味着这只军团失去了打开屏障的能力）。

因为阿尔萨斯急着去虐待炉石玩家，所以他把这个任务交给你。阿尔萨斯是一个经济实惠的人(?)，他希望到达银月城大厅的时间最短，请你帮他计算一下这个值是多少。

→ 最短路的应用—计数问题

P1144 最短路计数

题目描述：

给出一个 N 个顶点 M 条边的无向无权图，顶点编号为 $1 \sim N$ 。
问从顶点1开始，到其他每个点的最短路有几条。

最短路：So easy!

解决方法：

- 1、BFS
- 2、Dijkstra
- 3、SPFA

→ 最短路径计数问题

如何来求最短路径的条数呢？
相当于累计最优解的方案。

```
for (int i = edge[u].to; i < edge[u].to + edge[u].w; i++)  
    if (dist[v] == dist[u] + edge[i].w)  
        cnt[v] = (cnt[u] + cnt[v]) % 100003;  
    else if (!vis[v] && dist[v] > dist[u] + edge[i].w)  
    {  
        dist[v] = dist[u] + edge[i].w;  
        cnt[v] = cnt[u];  
    }
```


→ 最短路的应用—计数问题

P1608

“RP餐厅”的员工素质就是不一般，在齐刷刷的算出同一个电话号码之后，就准备让HZH,TZY去送快餐了，他们将自己居住的城市画了一张地图，已知在他们的地图上，有N个地方，而且他们目前处在标注为“1”的小镇上，而送餐的地点在标注为“N”的小镇。（有点废话）除此之外还知道这些道路都是单向的，从小镇i到j需要花费D[i,j]的时间，为了更高效快捷的将快餐送到顾客手中，

他们想走一条从小镇1到小镇N花费最少的一条路，但是他们临出发前，撞到因为在路上堵车而生气的FYY，深受启发，不能仅知道一条路线，万一...于是，他们邀请FYY一起来研究起了下一个问题：这个最少花费的路径有多少条？

对于100%的数据 $1 \leq N \leq 2000, 0 \leq E \leq N*(N-1), 1 \leq C \leq 10$.

普通Dijkstra + 去除重边



→ 最短路的应用—计数问题

P1608升级版

对于100%的数据：

$1 \leq N \leq 200000, 0 \leq E \leq 500000, 1 \leq C \leq 10.$

优先队列Dijkstra + 去除重边

NOI

→ 最短路的应用--差分约束

求下面不等式组的一组解：

$$X1 - X2 \leq 0$$

$$X1 - X5 \leq -1$$

$$X2 - X5 \leq 1$$

$$X3 - X1 \leq 5$$

$$X4 - X1 \leq 4$$

$$X4 - X3 \leq -1$$

$$X5 - X3 \leq -3$$

$$X5 - X4 \leq -3$$

$$Xi \leq 0 (i=1,2,3,4,5)$$

求完最短路后图中每条边

$$\text{dis}(v) \leq \text{dis}(u) + w(u, v)$$

对于不等式 $X_i - X_j \leq c$ ，把它化成不等式： $X_i \leq X_j + c$ ，就可以化成边 $V_j \rightarrow V_i$ ，权值为 c 。

最后，我们在这张图上求一次单源最短路径，这些不等式就会全部都满足

→ 最短路的应用--差分约束

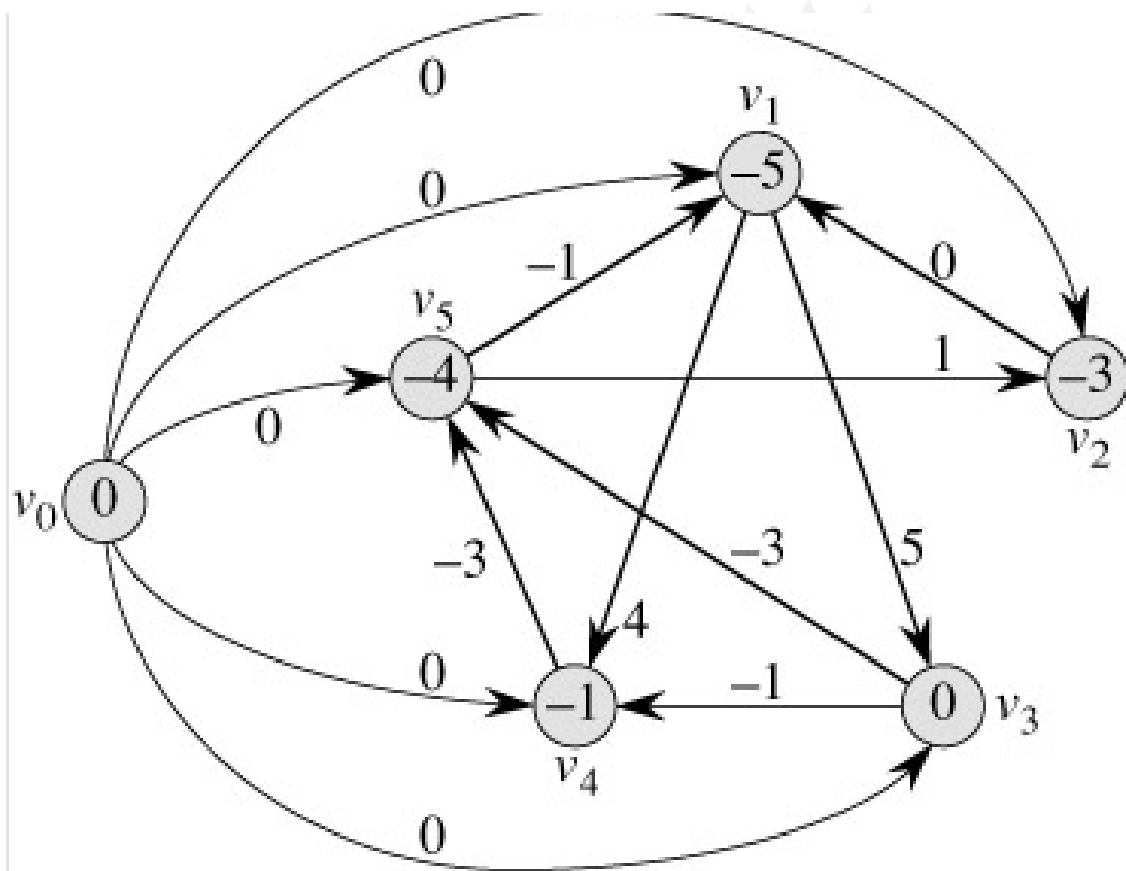
求下面不等式组的一组解：

$X_1 - X_2 \leq 0$ 对于最后一个 $X_i \leq 0$ 怎么办？
 $X_1 - X_5 \leq -1$ 新添一个 $X_0 = 0$, 并使 $X_i - X_0 \leq 0$!
 $X_2 - X_5 \leq 1$
 $X_3 - X_1 \leq 5$ 最后以 X_0 为源点，跑最短路
 $X_4 - X_1 \leq 4$
 $X_4 - X_3 \leq -1$ 最后每个点的 dis 值就为答案
 $X_5 - X_3 \leq -3$ 注意：由于有负边权，不能
 $X_5 - X_4 \leq -3$ 使用 $dijkstra$

$X_i \leq 0 (i=1,2,3,4,5)$

→ 最短路的应用--差分约束

最后的图及答案：



$$X_1 = -5$$

$$X_2 = -3$$

$$X_3 = 0$$

$$X_4 = -1$$

$$X_5 = -4$$



➔ 有限制的最短路

题目描述:

给出一个 N 个顶点 M 条边的无向无权图，顶点编号为 $1 \sim N$ 。问从顶点 1 开始，到顶点 N ，求经过奇数条边的最短路径和偶数条边的最短路径。



→ P2296 寻找道路(NOIP2014)

✧ 题目描述

在有向图G中，每条边的长度均为1，现给定起点和终点，请你在图中找一条从起点到终点的路径，该路径满足以下条件：

1. 路径上的所有点的出边所指向的点都直接或间接与终点连通。

2. 在满足条件1的情况下使路径最短。

注：图G中可能存在**重边和自环**，**题目保证终点没有出边**，请你输出符合条件的路径的长度。

→ P4162 [SCOI2009]最长距离

windy有一块矩形土地，被分为 $N*M$ 块 $1*1$ 的小格子。有的格子含有障碍物。如果从格子A可以走到格子B，那么两个格子的距离就为两个格子中心的欧几里德距离。如果从格子A不可以走到格子B，就没有距离。如果格子X和格子Y有公共边，并且X和Y均不含有障碍物，就可以从X走到Y。如果windy可以移走T块障碍物，求所有格子间的最大距离。保证移走T块障碍物以后，至少有一个格子不含有障碍物。

→ P1462 通往奥格瑞玛的道路

题目描述

在艾泽拉斯，有 n 个城市。编号为 $1, 2, 3, \dots, n$ 。
城市之间有 m 条双向的公路，连接着两个城市，从某个城市到另一个城市，会遭到联盟的攻击，进而损失一定的血量。

每次经过一个城市，都会被收取一定的过路费(包括起点和终点)。路上并没有收费站。假设1为暴风城， n 为奥格瑞玛，而他的血量最多为 b ，出发时他的血量是满的。

不希望花很多钱，他想知道，在可以到达奥格瑞玛的情况下，他所经过的所有城市中最多的一次收取的费用的最小值是多少。



➔ 最短路径练习题目

- 洛谷 P1135 奇怪的电梯
- 洛谷 P1576 最小花费
- 洛谷 P1938
- 洛谷 P3110
- 洛谷 P1491
- 洛谷 P1144 最短路径计数
- bzojP1295: [SCOI2009]最长距离 【 spfa变形 】

The background features a central point from which several rays of varying colors (blue, orange, red, and grey) extend outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two data series is visible on the right side.

Thank you!